```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert at beginning
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
```

Language

Run Debug Stop Share Save {} Beautify Language C

main.c

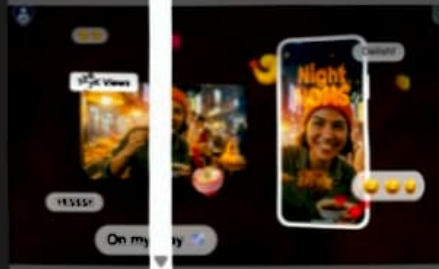
```c
// Insert at end
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Insert at position
void insertAtPosition(struct Node** head, int data, int position) {
    if (position == 0) {
        insertAtBeginning(head, data);
        return;
    }
```

Debug Stop Share Save {} Beautify Language C

main.c

```c
    struct Node* newNode = createNode(data);
    struct Node* temp = *head;

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range\n");
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

// Delete from beginning
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) return;

    struct Node* temp = *head;
    *head = (*head)->next;
```

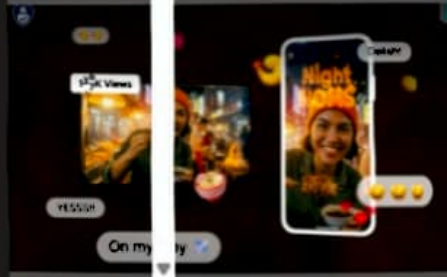
close ad [x]



main.c

```c
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) return;

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }

    struct Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
}

// Delete from position
void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;

    struct Node* temp = *head;
```

close ad [x]



main.c

Language C

```c
        head = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }

    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

// Traverse list
void traverse(struct Node* head) {
    struct Node* temp = head;
```

close ad [x]

Serve up tasty sounds.

main.c

```c
// Search element
int search(struct Node* head, int target) {
    struct Node* temp = head;

    while (temp != NULL) {
        if (temp->data == target)
            return 1;
        temp = temp->next;
    }
    return 0;
}

// Main function
int main() {
    struct Node* head = NULL;

    // Insert operations
    insertAtBeginning(&head, 10);
    insertAtBeginning(&head, 5);
    insertAtEnd(&head, 20);
    insertAtPosition(&head, 15, 2);

```

close ad [x]
Serve up tasty sounds.

Run Debug Stop Share Save { } Beautify Language C

main.c

```c
    // Delete operations
    deleteFromBeginning(&head);
    printf("After deleting from beginning:\n");
    traverse(head);

    deleteFromEnd(&head);
    printf("After deleting from end:\n");
    traverse(head);

    deleteFromPosition(&head, 1);
    printf("After deleting from position 1:\n");
    traverse(head);

    // Search
    int key = 15;
    if (search(head, key))
        printf("%d found in list\n", key);
    else
        printf("%d not found in list\n", key);

    return 0;
}
```

close ad [x]

(5) WhatsApp | Online C Compiler - online edit | Linked List Operations | Ask Gemini

onlinegdb.com/online_c_compiler

OnlineGDB
online compiler and debugger for c/c++

Welcome, **Gaanashri**

- Create New Project
- My Projects
- Classroom new
- Learn Programming
- Programming Questions
- Upgrade
- Logout

Run | Debug | Stop | Share | Save | { } Beautify | Language C

main.c

```c
    // Delete operations
    deleteFromBeginning(&head);
    printf("After deleting from beginning:\n");
```

close ad [x]

input

```
Linked List after insertions:
5 -> 10 -> 15 -> 20 -> NULL
After deleting from beginning:
10 -> 15 -> 20 -> NULL
After deleting from end:
10 -> 15 -> NULL
After deleting from position 1:
10 -> NULL
15 not found in list


...Program finished with exit code 0
Press ENTER to exit console.
```